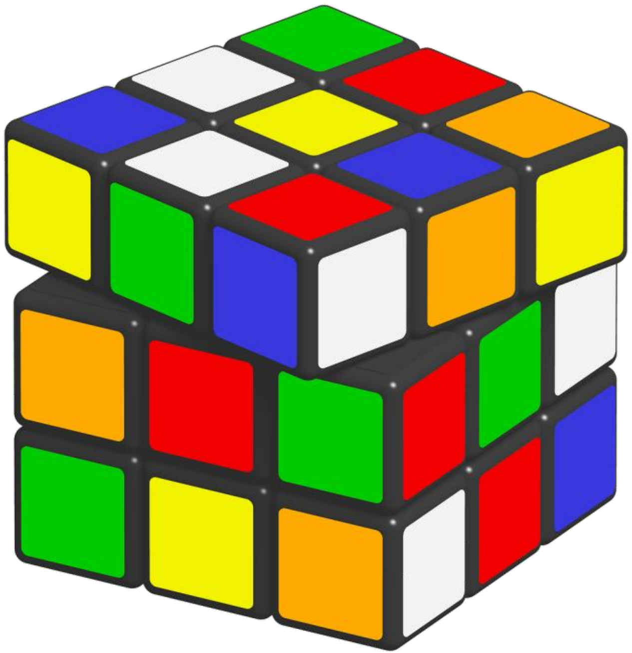
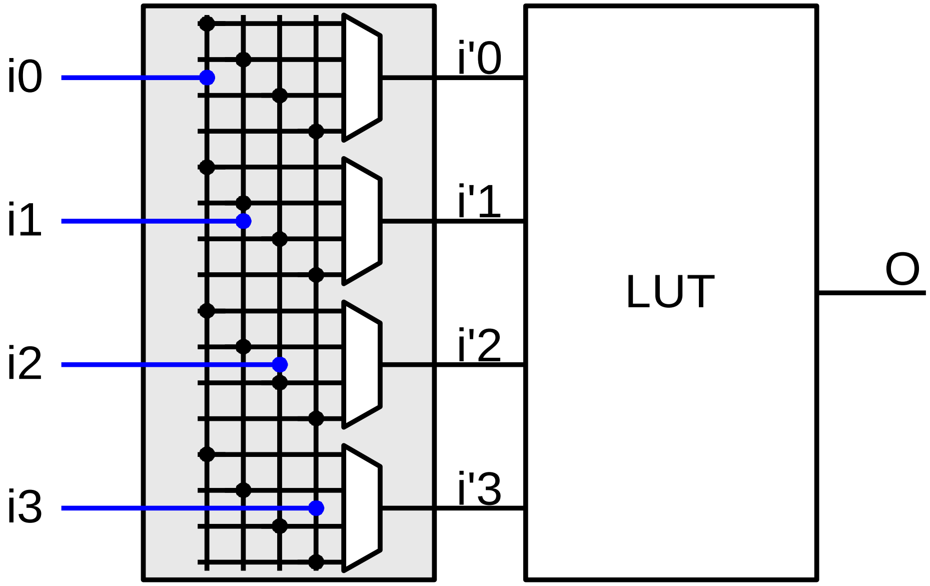
Architecture and CAD for FPGAs – ACF

Exercise 1 – LUT input pin-swapping

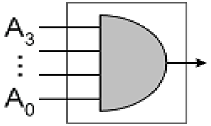
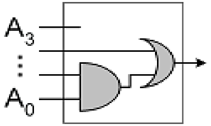
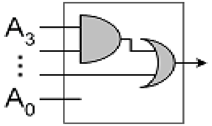
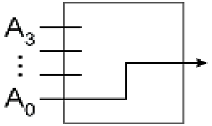
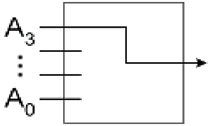


Exercise – LUT input pin-swapping

- **LUT pin-swapping implements a virtual input crossbar to the LUT**
  → helps during routing
- **Questions:**
  - How does it work?
  - How do we permute the LUT entries?
  - Are there limits? (situation where we cannot use this)
  - Are there other places where we can use this?



Look-up Tables – LUTs as function generators

A_3 A_2 A_1 A_0									
0:	0	0	0	0	0	0	0	0	0
1:	0	0	0	1	0	0	0	1	0
2:	0	0	1	0	0	0	1	0	0
3:	0	0	1	1	0	1	1	1	0
4:	0	1	0	0	0	1	0	0	0
5:	0	1	0	1	0	1	0	1	0
6:	0	1	1	0	0	1	1	0	0
7:	0	1	1	1	0	1	1	1	0
8:	1	0	0	0	0	0	0	0	1
9:	1	0	0	1	0	0	0	1	1
A:	1	0	1	0	0	0	1	0	1
B:	1	0	1	1	0	1	1	1	1
C:	1	1	0	0	0	1	1	0	1
D:	1	1	0	1	0	1	1	1	1
E:	1	1	1	0	0	1	1	0	1
F:	1	1	1	1	1	1	1	1	1

Exercise – LUT input pin-swapping

- **How do we permute the LUT entries?**
- Swapping LUT inputs is just a permuting address bits to the LUT table
- We only have to copy bit-by-bit the table using the swapped address bit:

`for i in 0000 to 1111 loop`:

`LUT(`$\mathbf{i_3', i_2', i_1', i_0'}$**`)' = LUT (i)`**

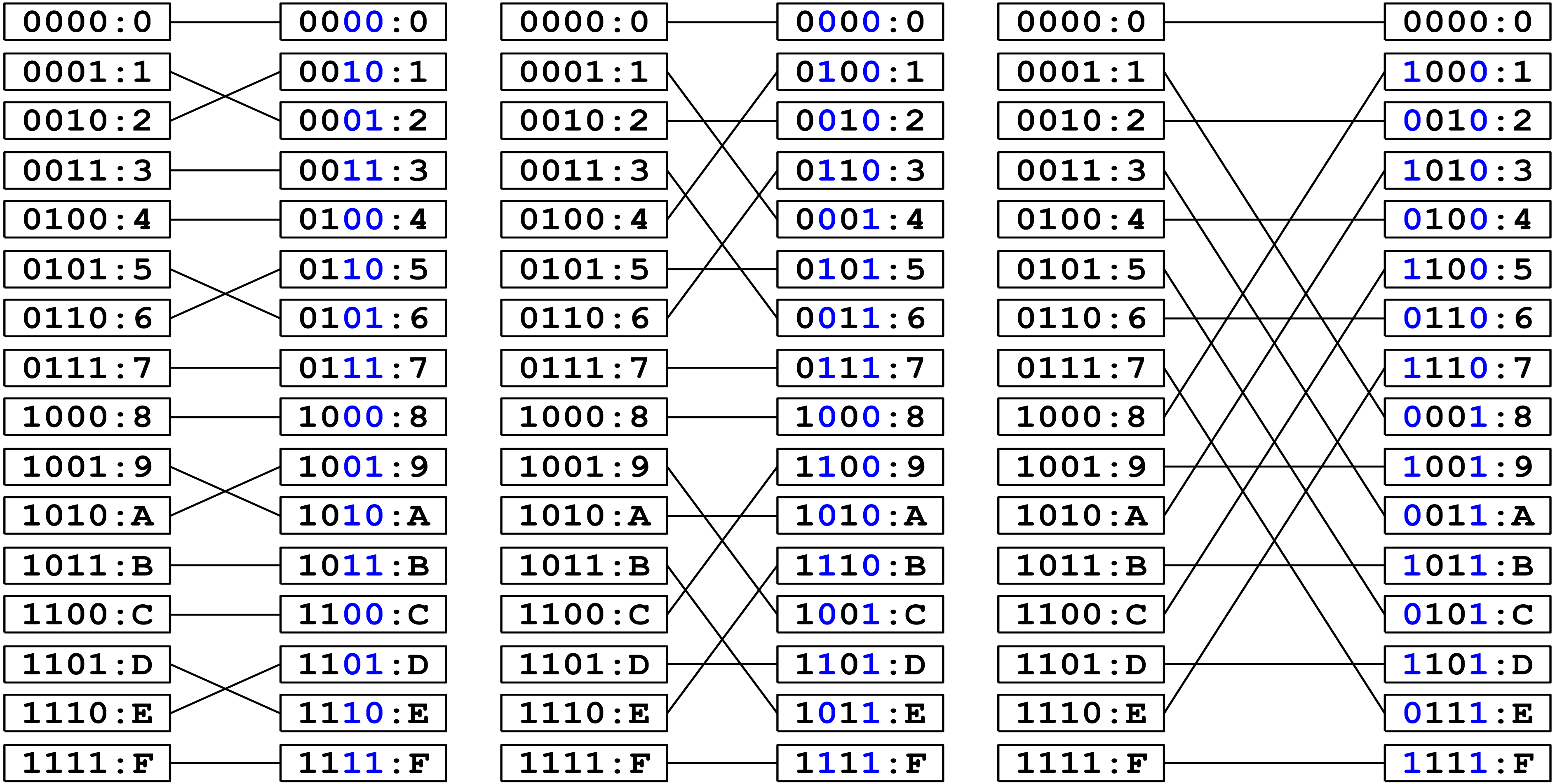
e.g., if we want to swap $\mathbf{i_0}$ with $\mathbf{i_1}$, then

$\mathbf{i_0' = i_1}$ and $\mathbf{i_1' = i_0}$

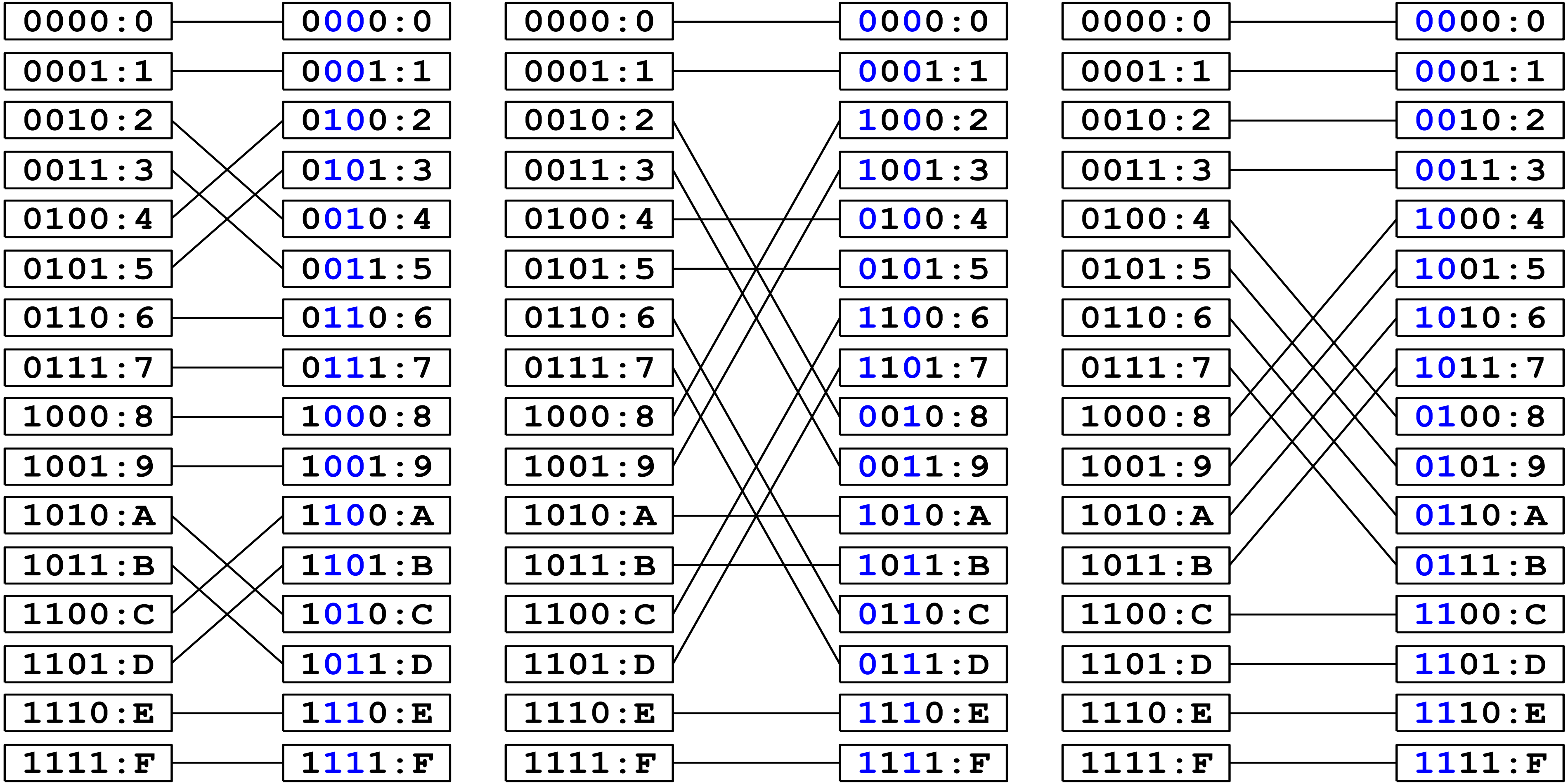
- Can also be done by shifting (e.g., when swapping only two inputs)

1	A_3, A_2, A_1, A_0	LUT-value AND gate	LUT-value OR gate
0	0000	0	0
1	0001	0	1
2	0010	0	1
3	0011	0	1
4	0100	0	1
5	0101	0	1
6	0110	0	1
7	0111	0	1
8	1000	0	1
9	1001	0	1
A	1010	0	1
B	1011	0	1
C	1100	0	1
D	1101	0	1
E	1110	0	1
F	1111	1	1

Exercise – LUT input pin-swapping

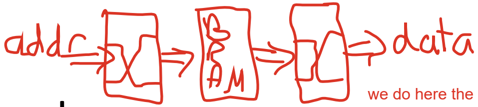


Exercise – LUT input pin-swapping



Exercise – LUT input pin-swapping

- **Are there limits?**
  (situation where we cannot use this)
- If the LUT is supporting distributed memory (like the shift-register or register-file mode), LUT inputs are addresses that cannot be swapped
- **Are there other places where we can use this?**
- BRAMs in ROM mode
  → can be arbitrarily used for address and data bus
  (Xilinx does not do this)



we do here the permutation directly

- Does not work in RAM mode
  → we use usually use different read and write address ports and data is separated anyway
  (swapping creates a dependency with another port)

1	A_3, A_2, A_1, A_0	LUT-value AND gate	LUT-value OR gate
0	0000	0	0
1	0001	0	1
2	0010	0	1
3	0011	0	1
4	0100	0	1
5	0101	0	1
6	0110	0	1
7	0111	0	1
8	1000	0	1
9	1001	0	1
A	1010	0	1
B	1011	0	1
C	1100	0	1
D	1101	0	1
E	1110	0	1
F	1111	1	1

Exercise – dependent LUT inputs

- **Write a function/program that gets in an init string and that returns the dependent variables.**

  For instance, if someone has implemented a 3-input Boolean function in our LUT-4, you should detect which LUT input is unused

- **For each bit index, we have to compare pairwise (by setting the bit index once 0 and once 1) if the LUT value is identical for all bit positions given by the index formed by the remaining bits (see figure)**

```
# 0  0-+                0-+        0-+     0-+
#      |                  |          |       |
# 1  0-|-+              0-|-+      0-|-+   1-+
#      | |                | |        | |
# 2  0-|-|-+            0-|-|-+    1-+ |   0-+
#      | | |              | | |        |       |
# 3  0-|-|-|-+          0-|-|-|-+  1-+-|   1-+
#      | | | |            | | | |
# 4  0-|-|-|-|-+        1-+ | | |  0-+    0-+
#      | | | | |            | | |    |      |
# 5  0-|-|-|-|-|-+      1---+ | |  0-|-+  1-+
#      | | | | | |            | |    | |
# 6  0-|-|-|-|-|-|-+    1-----+ |  1-+ |  0-+
#      | | | | | | |            |      |      |
# 7  0-|-|-|-|-|-|-|-+  1-------+  1-+-|  1-+
#      | | | | | | | |
# 8  1-+ | | | | | | |  0-+        0-+    0-+
#        | | | | | | |    |          |      |
# 9  1---+ | | | | | |  0-|-+      0-|-+  1-+
#          | | | | | |    | |        | |
# A  1-----+ | | | | |  0-|-|-+    1-+ |  0-+
#            | | | | |    | | |        |      |
# B  1-------+ | | | |  0-|-|-|-+  1-+-|  1-+
#              | | | |    | | | |
# C  1---------+ | | |  1-+ | | |  0-+    0-+
#                | | |      | | |    |      |
# D  1-----------+ | |  1---+ | |  0-|-+  1-+
#                  | |        | |    | |
# E  1-------------+ |  1-----+ |  1-+ |  0-+
#                    |          |      |      |
# F  1---------------|  1-------+  1-+-|  1-+
```

Any Questions?